

LEVEL 1: BASICS

1. Retrieve customer names and emails for email Marketing.

SELECT NAME, EMAIL FROM CUSTOMERS;

2238 • **SELECT NAME, EMAIL FROM CUSTOMERS;**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	NAME	EMAIL
▶	Thomas Owens	user1@example.com
	Charles Grant	user2@example.com
	Kaitlin Richards	user3@example.com
	Christina Williams	user4@example.com
	David Allen	user5@example.com
	Mark Duke	user6@example.com
	Briana Wright	user7@example.com
	John Bryan	user8@example.com
	Jason Thompson	user9@example.com

CUSTOMERS 9 x

2. View complete product catalog with all available details.

```
SELECT * FROM PRODUCTS;
```

2242 • **SELECT * FROM PRODUCTS;**

product_id	name	category	price	stock_quantity	added_on
1	Plant No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
9	Common Study	Toys	985.19	171	2023-07-20 13:06:42

PRODUCTS 10 x

3. List all unique product categories.

SELECT DISTINCT CATEGORY FROM PRODUCTS;

2246 • **SELECT DISTINCT CATEGORY FROM PRODUCTS;**

Result Grid |  |  Filter Rows: | Export:  | Wrap Cell Content: 

	CATEGORY
▶	Home
	Clothing
	Electronics
	Toys
	Books

4. Show all products priced above ₹1,000.

SELECT * FROM PRODUCTS WHERE PRICE>1000;

2250 • **SELECT * FROM PRODUCTS WHERE PRICE>1000;**

Result Grid   Filter Rows: **Edit:**    **Export/Import:**   **Wrap Cell Content:** 

	product_id	name	category	price	stock_quantity	added_on
▶	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06

PRODUCTS 13 x

5. Display products within a mid-range price bracket (₹2,000 to ₹5,000).

```
SELECT * FROM PRODUCTS WHERE PRICE BETWEEN 2000 AND 5000;
```

2254 • **SELECT * FROM PRODUCTS WHERE PRICE BETWEEN 2000 AND 5000;**

	product_id	name	category	price	stock_quantity	added_on
▶	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38

PRODUCTS 14 x

6. Fetch data for specific customer IDs (e.g., from loyalty program list).

```
SELECT * FROM ORDERS WHERE CUSTOMER_ID IN (6,18);
```

2258 • **SELECT * FROM ORDERS WHERE CUSTOMER_ID IN (6,18);**

	order_id	customer_id	order_date	status	total_amount
▶	9	6	2025-06-10 17:00:25	Pending	24219.20
	24	6	2024-06-19 07:09:04	Pending	22882.19
	50	6	2024-09-04 14:45:56	Delivered	14204.67
	96	6	2025-01-10 04:00:48	Shipped	1537.38
	106	6	2024-07-16 16:57:13	Pending	11486.99
	172	6	2024-09-17 03:06:14	Shipped	16661.54
	180	6	2024-06-21 17:29:54	Shipped	2680.68
	194	6	2024-09-23 13:29:00	Shipped	296.17
	261	6	2025-02-11 04:22:55	Delivered	1447.94

ORDERS 15 x

7. Identify customers whose names start with the letter 'A'.

```
SELECT * FROM CUSTOMERS WHERE NAME LIKE 'A%';
```

2262 • SELECT * FROM CUSTOMERS WHERE NAME LIKE 'A%';

Result Grid | | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

	customer_id	name	email	phone	created_at
▶	15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
	16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
	19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
	27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29
•	NUL	NUL	NUL	NUL	NUL

CUSTOMERS 16 ✕

8. List electronics products priced under ₹3,000.

SELECT * FROM PRODUCTS WHERE CATEGORY='ELECTRONICS' AND PRICE<3000;

[illegible]

9. Display product names and prices in descending order of price.

SELECT NAME, PRICE FROM PRODUCTS ORDER BY PRICE DESC;

2270 • **SELECT NAME, PRICE FROM PRODUCTS ORDER BY PRICE DESC;**

NAME	PRICE
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68

PRODUCTS 18 x

10. Display product names and prices, sorted by price and then by name.

SELECT NAME, PRICE FROM PRODUCTS ORDER BY PRICE, NAME;

2274 • **SELECT NAME, PRICE FROM PRODUCTS ORDER BY PRICE, NAME;**

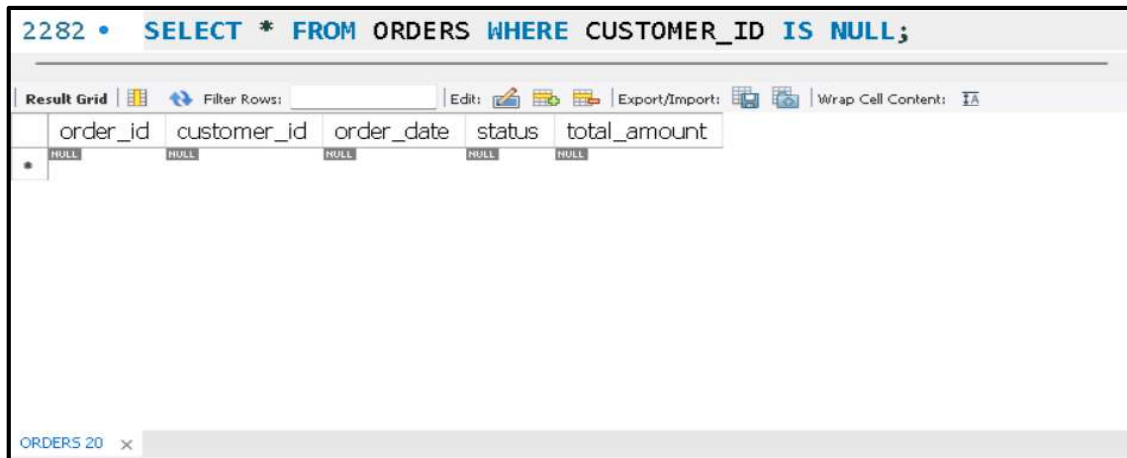
NAME	PRICE
If Whatever	177.40
Listen Development	296.17
Actually Term	396.11
Television Stock	421.73
Southern Thing	512.46
Plant No	639.43
Place Low	723.97
Stock Article	834.75
Toward Minute	857.85

PRODUCTS 19 x

LEVEL 2: FILTERING AND FORMATING

1. Retrieve orders where customer information is missing (possibly due to data migration or deletion).

```
SELECT * FROM ORDERS WHERE CUSTOMER_ID IS NULL;
```



2282 • **SELECT * FROM ORDERS WHERE CUSTOMER_ID IS NULL;**

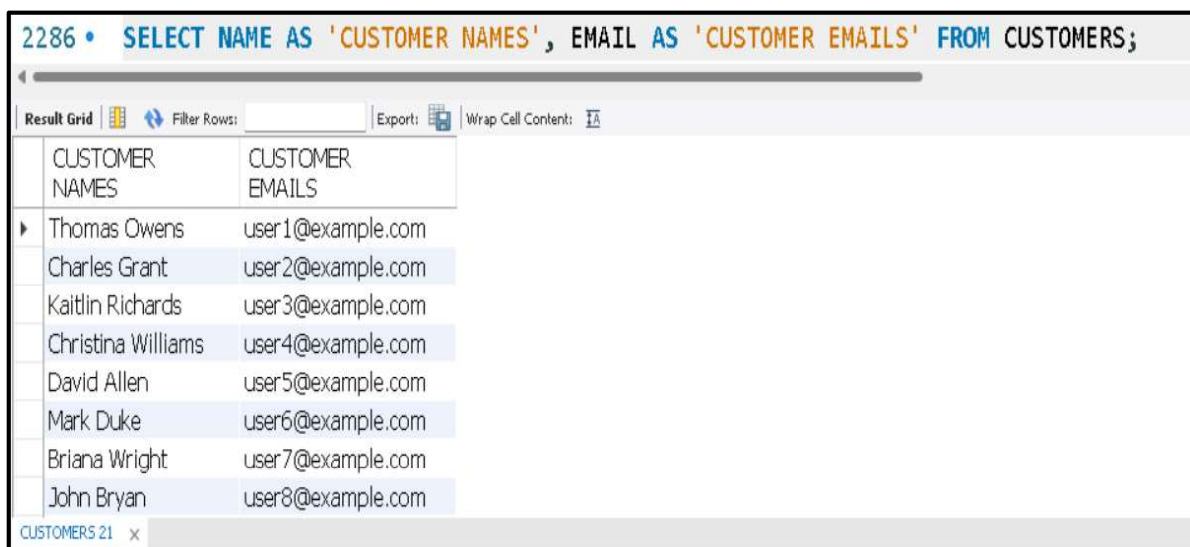
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	order_id	customer_id	order_date	status	total_amount
*	NULL	NULL	NULL	NULL	NULL

ORDERS 20 x

2. Display customer names and emails using column aliases for frontend readability.

```
SELECT NAME AS 'CUSTOMER NAMES', EMAIL AS 'CUSTOMER EMAILS' FROM CUSTOMERS;
```



2286 • **SELECT NAME AS 'CUSTOMER NAMES', EMAIL AS 'CUSTOMER EMAILS' FROM CUSTOMERS;**

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	CUSTOMER NAMES	CUSTOMER EMAILS
▶	Thomas Owens	user1@example.com
	Charles Grant	user2@example.com
	Kaitlin Richards	user3@example.com
	Christina Williams	user4@example.com
	David Allen	user5@example.com
	Mark Duke	user6@example.com
	Briana Wright	user7@example.com
	John Bryan	user8@example.com

CUSTOMERS 21 x

3. Calculate total value per item ordered by multiplying quantity and item price.

SELECT ORDER_ID, PRODUCT_ID, QUANTITY, ITEM_PRICE, (QUANTITY * ITEM_PRICE) AS 'TOTAL VALUE' FROM ORDER_ITEMS;

2290 • **SELECT ORDER_ID,PRODUCT_ID,QUANTITY,ITEM_PRICE,(QUANTITY * ITEM_PRICE)**
2291 **AS 'TOTAL VALUE' FROM ORDER_ITEMS;**

	ORDER_ID	PRODUCT_ID	QUANTITY	ITEM_PRICE	TOTAL VALUE
▶	1	19	2	4707.14	9414.28
	2	6	3	177.40	532.20
	3	9	3	985.19	2955.57
	3	23	1	2208.99	2208.99
	4	35	2	4734.89	9469.78
	5	19	1	4707.14	4707.14
	5	7	2	4897.36	9794.72
	6	7	3	4897.36	14692.08

Result 23 x

4. Combine customer name and phone number in a single column.

SELECT CONCAT (NAME, ' - ', PHONE) AS 'CUSTOMER DETAILS' FROM CUSTOMERS;

2295 • **SELECT CONCAT(NAME, ' - ',PHONE) AS 'CUSTOMER DETAILS' FROM CUSTOMERS;**

	CUSTOMER DETAILS
▶	Thomas Owens - 142-479-1945
	Charles Grant - 9153947511
	Kaitlin Richards - 2073473421
	Christina Williams - 586-605-5...
	David Allen - (751)456-8289x1
	Mark Duke - (144)957-2811
	Briana Wright - 223-833-9635
	John Bryan - 045.568.0798x27
	Jason Thompson - 1862659420

Result 24 x

5. Extract only the date part from order timestamps for date-wise reporting.

`SELECT ORDER_ID, DATE(ORDER_DATE) AS 'ONLY ORDER DATE' FROM ORDERS;`

2299 • `SELECT ORDER_ID, DATE(ORDER_DATE) AS 'ONLY ORDER DATE' FROM ORDERS;`

ORDER_ID	ONLY ORDER DATE
1	2025-03-02
2	2024-10-09
3	2025-05-08
4	2024-09-19
5	2025-04-08
6	2024-10-25
7	2024-07-29
8	2024-07-30

Result 25 x

6. List products that do not have any stock left.

`SELECT * FROM PRODUCTS WHERE STOCK_QUANTITY IS NULL;`

2303 • `SELECT * FROM PRODUCTS WHERE STOCK_QUANTITY IS NULL;`

product_id	name	category	price	stock_quantity	added_on
*	NULL	NULL	NULL	NULL	NULL

PRODUCTS 26 x

LEVEL 3: AGGREGATIONS

1. Count the total number of orders placed.

`SELECT COUNT (*) AS 'TOTAL ORDERS' FROM ORDERS;`

2311 • <code>SELECT COUNT(*) AS 'TOTAL ORDERS' FROM ORDERS;</code>	
Result Grid	Filter Rows: Export: Wrap Cell Content:
TOTAL ORDERS	
▶ 400	
Result 27 x	

2. Calculate the total revenue collected from all orders.

`SELECT SUM(AMOUNT_PAID) AS 'TOTAL REVENUE' FROM PAYMENTS;`

2315 • <code>SELECT SUM(AMOUNT_PAID) AS 'TOTAL REVENUE' FROM PAYMENTS;</code>	
Result Grid	Filter Rows: Export: Wrap Cell Content:
TOTAL REVENUE	
▶ 6960973.66	
Result 28 x	

3. Calculate the average order value.

```
SELECT AVG(AMOUNT_PAID) AS 'AVERAGE ORDER VALUE' FROM PAYMENTS;
```

2319 • SELECT AVG(AMOUNT_PAID) AS 'AVERAGE ORDER VALUE' FROM PAYMENTS;	
Result Grid	Filter Rows: Export: Wrap Cell Content:
AVERAGE ORDER VALUE	
▶ 17402.434150	
Result 29 x	

4. Count the number of customers who have placed at least one order.

```
SELECT COUNT (DISTINCT CUSTOMER_ID) AS 'ACTIVE CUSTOMERS' FROM ORDERS;
```

2323 • SELECT COUNT(DISTINCT CUSTOMER_ID) AS 'ACTIVE CUSTOMERS' FROM ORDERS;	
Result Grid	Filter Rows: Export: Wrap Cell Content:
ACTIVE CUSTOMERS	
▶ 30	
Result 30 x	

5. Find the number of orders placed by each customer.

```
SELECT CUSTOMER_ID, COUNT (*) AS 'NO. OF ORDER PLACED' FROM ORDERS GROUP BY CUSTOMER_ID;
```

2327 • `SELECT CUSTOMER_ID, COUNT (*) AS 'NO. OF ORDER PLACED' FROM ORDERS GROUP BY CUSTOMER_ID;`

CUSTOMER_ID	NO. OF ORDER PLACED
1	12
2	17
3	17
4	9
5	14
6	16
7	13
8	10

Result 31 x

6. Find total sales amount made by each customer.

```
SELECT CUSTOMER_ID, SUM(TOTAL_AMOUNT) AS 'TOTAL SALES PER CUSTOMER' FROM ORDERS GROUP BY CUSTOMER_ID;
```

2331 • `SELECT CUSTOMER_ID, SUM(TOTAL_AMOUNT) AS 'TOTAL SALES PER CUSTOMER'`
2332 `FROM ORDERS GROUP BY CUSTOMER_ID;`

CUSTOMER_ID	TOTAL SALES PER CUSTOMER
1	183747.44
2	284420.07
3	253783.31
4	137562.22
5	262504.19
6	212173.58
7	167960.11
8	164701.78

Result 32 x

7. List the number of products sold per category.

```
SELECT P. CATEGORY, SUM (OI. QUANTITY) AS 'TOTAL PRODUCTS SOLD' FROM Products P  
JOIN ORDER_ITEMS OI ON P. PRODUCT_ID=OI.PRODUCT_ID GROUP BY P. CATEGORY;
```

2336 • `SELECT P.CATEGORY, SUM(OI.QUANTITY) AS 'TOTAL PRODUCTS SOLD' FROM Products P`
2337 `JOIN ORDER_ITEMS OI ON P.PRODUCT_ID=OI.PRODUCT_ID GROUP BY P.CATEGORY;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [FA](#)

	CATEGORY	TOTAL PRODUCTS SOLD
▶	Home	443
	Clothing	559
	Electronics	687
	Toys	405
	Books	350

Result 33 x

8. Find the average item price per category.

```
SELECT CATEGORY, AVG(PRICE) AS 'AVERAGE PRICE' FROM PRODUCTS GROUP BY  
CATEGORY;
```

2341 • `SELECT CATEGORY, AVG(PRICE) AS 'AVERAGE PRICE' FROM PRODUCTS GROUP BY CATEGORY;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [FA](#)

	CATEGORY	AVERAGE PRICE
▶	Home	2146.367500
	Clothing	3434.581538
	Electronics	2653.388571
	Toys	2516.526250
	Books	3167.084286

Result 34 x

9. Show number of orders placed per day.

```
SELECT DATE(ORDER_DATE) AS 'ORDER_DAY', COUNT(*) AS 'TOTAL_ORDERS' FROM  
ORDERS  
GROUP BY DATE(ORDER_DATE);
```

2345 • `SELECT DATE(ORDER_DATE) AS 'ORDER_DAY', COUNT(*) AS 'TOTAL_ORDERS'`
2346 `FROM ORDERS GROUP BY DATE(ORDER_DATE);`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

	ORDER_DAY	TOTAL_ORDERS
▶	2025-03-02	3
	2024-10-09	2
	2025-05-08	2
	2024-09-19	3
	2025-04-08	3
	2024-10-25	3
	2024-07-29	2
	2024-07-30	2
	2025-06-10	2

Result 35 x

10. List total payments received per payment method.

```
SELECT METHOD, SUM(AMOUNT_PAID) AS 'TOTAL PAYMENTS' FROM PAYMENTS GROUP  
BY METHOD;
```

2350 • `SELECT METHOD, SUM(AMOUNT_PAID) AS 'TOTAL PAYMENTS' FROM PAYMENTS GROUP BY METHOD;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	METHOD	TOTAL PAYMENTS
▶	Credit Card	1754603.10
	Net Banking	1658383.90
	UPI	1617408.78
	Debit Card	1930577.88

Result 36 x

LEVEL 4: MULTI-TABLE QUERIES (JOINS)

1. Retrieve order details along with the customer name (INNER JOIN).

```
SELECT O.*, C.NAME FROM ORDERS O INNER JOIN CUSTOMERS C ON O.  
CUSTOMER_ID=C. CUSTOMER_ID;
```

2358 • `SELECT O.*,C.NAME FROM ORDERS O INNER JOIN CUSTOMERS C ON O.CUSTOMER_ID=C.CUSTOMER_ID;`

	order_id	customer_id	order_date	status	total_amount	NAME
▶	14	1	2024-09-24 21:21:38	Shipped	15803.34	Thomas Owens
	17	1	2024-08-19 21:17:57	Pending	11173.77	Thomas Owens
	61	1	2024-12-26 23:21:58	Shipped	13053.00	Thomas Owens
	76	1	2025-01-14 22:59:54	Pending	23506.81	Thomas Owens
	92	1	2024-09-26 11:33:58	Shipped	834.75	Thomas Owens
	109	1	2025-03-17 04:56:18	Shipped	12190.14	Thomas Owens
	127	1	2025-06-10 18:01:32	Pending	20160.64	Thomas Owens
	135	1	2025-03-11 22:58:23	Pending	8891.79	Thomas Owens
	144	1	2024-09-19 09:18:22	Shipped	12093.54	Thomas Owens

Result 37 x

2. Get list of products that have been sold (INNER JOIN with order_items).

```
SELECT DISTINCT P. PRODUCT_ID, P.NAME FROM PRODUCTS P INNER JOIN  
ORDER_ITEMS OI ON OI. PRODUCT_ID=P.PRODUCT_ID;
```

2362 • `SELECT DISTINCT P.PRODUCT_ID,P.NAME FROM PRODUCTS P INNER JOIN ORDER_ITEMS OI`
2363 `ON OI.PRODUCT_ID=P.PRODUCT_ID;`

	PRODUCT_ID	NAME
▶	1	Plant No
	2	Population Social
	3	Available Answer
	4	Any Question
	5	Natural Network
	6	If Whatever
	7	Response Indeed
	8	Every Amount
	9	Common Study

Result 38 x

3. List all orders with their payment method (INNER JOIN).

```
SELECT O. ORDER_ID, PY. METHOD FROM ORDERS O INNER JOIN PAYMENTS PY ON O.  
ORDER_ID=PY.ORDER_ID;
```

2367 • **SELECT** O.ORDER_ID,PY.METHOD **FROM** ORDERS O **INNER JOIN** PAYMENTS PY
2368 **ON** O.ORDER_ID=PY.ORDER_ID;

ORDER_ID	METHOD
14	UPI
17	UPI
61	Net Banking
76	Credit Card
92	Net Banking
109	UPI
127	Debit Card
135	Debit Card
144	UPI

Result 39 x

4. Get list of customers and their orders (LEFT JOIN).

```
SELECT C. CUSTOMER_ID, C.name, O. ORDER_ID FROM CUSTOMERS C LEFT JOIN  
ORDERS O ON C. CUSTOMER_ID=O.CUSTOMER_ID;
```

2372 • **SELECT** C.CUSTOMER_ID,C.name,O.ORDER_ID **FROM** CUSTOMERS C **LEFT JOIN** ORDERS O
2373 **ON** C.CUSTOMER_ID=O.CUSTOMER_ID;

CUSTOMER_ID	name	ORDER_ID
1	Thomas Owens	14
1	Thomas Owens	17
1	Thomas Owens	61
1	Thomas Owens	76
1	Thomas Owens	92
1	Thomas Owens	109
1	Thomas Owens	127
1	Thomas Owens	135
1	Thomas Owens	144

Result 40 x

5. List all products along with order item quantity (LEFT JOIN).

```
SELECT P. PRODUCT_ID, P.NAME, OI. QUANTITY FROM PRODUCTS P LEFT JOIN  
ORDER_ITEMS OI ON P. PRODUCT_ID=OI.PRODUCT_ID;
```

2377 • **SELECT** P.PRODUCT_ID, P.NAME, OI.QUANTITY **FROM** PRODUCTS P **LEFT JOIN** ORDER_ITEMS OI
2378 **ON** P.PRODUCT_ID=OI.PRODUCT_ID;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

	PRODUCT_ID	NAME	QUANTITY
▶	1	Plant No	1
	1	Plant No	3
	1	Plant No	1
	1	Plant No	1
	1	Plant No	2
	1	Plant No	3
	1	Plant No	2
	1	Plant No	2
	1	Plant No	2

Result 41 x

6. List all payments including those with no matching orders (RIGHT JOIN).

```
SELECT O. ORDER_ID, PY. PAYMENT_ID, PY. AMOUNT_PAID, PY. METHOD FROM ORDERS O  
O RIGHT JOIN PAYMENTS PY ON O. ORDER_ID = PY. ORDER_ID;
```

2382 • **SELECT** O.ORDER_ID, PY.PAYMENT_ID, PY.AMOUNT_PAID, PY.METHOD **FROM** ORDERS O
2383 **RIGHT JOIN** PAYMENTS PY **ON** O.ORDER_ID = PY.ORDER_ID;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

	ORDER_ID	PAYMENT_ID	AMOUNT_PAID	METHOD
▶	1	1	9414.28	Credit Card
	2	2	532.20	Net Banking
	3	3	5164.56	Credit Card
	4	4	9469.78	UPI
	5	5	14501.86	UPI
	6	6	31050.17	UPI
	7	7	3043.67	UPI
	8	8	32714.06	Net Banking
	9	9	24219.20	Net Banking

Result 42 x

7. Combine data from three tables: customer, order and payment.

```
SELECT C.NAME, O. ORDER_ID, PY. AMOUNT_PAID FROM CUSTOMERS C INNER JOIN  
ORDERS O ON C. CUSTOMER_ID=O.CUSTOMER_ID INNER JOIN PAYMENTS PY ON O.  
ORDER_ID=PY.ORDER_ID;
```

2387 • `SELECT C.NAME, O.ORDER_ID, PY.AMOUNT_PAID FROM CUSTOMERS C INNER JOIN ORDERS O`
2388 `ON C.CUSTOMER_ID=O.CUSTOMER_ID INNER JOIN PAYMENTS PY ON O.ORDER_ID=PY.ORDER_ID;`

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
NAME	ORDER_ID	AMOUNT_PAID		
▶ Thomas Owens	14	15803.34		
Thomas Owens	17	11173.77		
Thomas Owens	61	13053.00		
Thomas Owens	76	23506.81		
Thomas Owens	92	834.75		
Thomas Owens	109	12190.14		
Thomas Owens	127	20160.64		
Thomas Owens	135	8891.79		
Thomas Owens	144	12093.54		

Result 43 x

LEVEL 5: SUBQUERIES (INNER QUERIES)

1. List all products priced above the average product price.

SELECT * FROM PRODUCTS WHERE PRICE > (SELECT AVG(PRICE) FROM PRODUCTS);

2396 • **SELECT * FROM PRODUCTS WHERE PRICE > (SELECT AVG(PRICE) FROM PRODUCTS);**

	product_id	name	category	price	stock_quantity	added_on
▶	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
	19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01

PRODUCTS 44 x

2. Find customers who have placed at least one order.

SELECT * FROM CUSTOMERS WHERE CUSTOMER_ID IN (SELECT CUSTOMER_ID FROM ORDERS);

2400 • **SELECT * FROM CUSTOMERS WHERE CUSTOMER_ID IN (SELECT CUSTOMER_ID FROM ORDERS);**

	customer_id	name	email	phone	created_at
▶	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
	2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
	3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
	4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
	5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
	6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
	7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
	8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
	9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51

CUSTOMERS 45 x

3. Show orders whose total amount is above the average for that customer.

```
SELECT * FROM ORDERS O WHERE TOTAL_AMOUNT > (SELECT AVG(TOTAL_AMOUNT)
FROM ORDERS WHERE CUSTOMER_ID=O.CUSTOMER_ID);
```

2404 • `SELECT * FROM ORDERS O WHERE TOTAL_AMOUNT > (SELECT AVG(TOTAL_AMOUNT) FROM ORDERS`
2405 `WHERE CUSTOMER_ID=O.CUSTOMER_ID);`

	order_id	customer_id	order_date	status	total_amount
▶	6	29	2024-10-25 07:33:59	Cancelled	31050.17
	8	19	2024-07-30 22:49:49	Cancelled	32714.06
	9	6	2025-06-10 17:00:25	Pending	24219.20
	10	28	2025-02-16 12:45:59	Delivered	24342.52
	14	1	2024-09-24 21:21:38	Shipped	15803.34
	16	5	2024-09-09 03:00:40	Shipped	22856.73
	18	13	2024-06-15 12:57:04	Cancelled	32001.24
	21	22	2025-03-12 18:53:46	Pending	25364.11
	22	20	2025-02-21 11:21:06	Shipped	21281.44

ORDERS 47 x

4. Display customers who haven't placed any orders.

```
SELECT * FROM CUSTOMERS WHERE CUSTOMER_ID NOT IN (SELECT CUSTOMER_ID
FROM ORDERS);
```

2409 • `SELECT * FROM CUSTOMERS WHERE CUSTOMER_ID NOT IN (SELECT CUSTOMER_ID FROM ORDERS);`

	customer_id	name	email	phone	created_at
*	NULL	NULL	NULL	NULL	NULL

CUSTOMERS 48 x

5. Show products that were never ordered.

```
SELECT * FROM PRODUCTS WHERE PRODUCT_ID NOT IN (SELECT PRODUCT_ID FROM ORDER_ITEMS);
```

2413 • **SELECT * FROM PRODUCTS WHERE PRODUCT_ID NOT IN (SELECT PRODUCT_ID FROM ORDER_ITEMS);**

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	product_id	name	category	price	stock_quantity	added_on
*	NULL	NULL	NULL	NULL	NULL	NULL

PRODUCTS 51 x

6. Show highest value order per customer.

```
SELECT O. ORDER_ID, O. CUSTOMER_ID, O. TOTAL_AMOUNT AS 'HIGHEST ORDER VALUE' FROM ORDERS O
```

```
WHERE TOTAL_AMOUNT = (SELECT MAX(TOTAL_AMOUNT) FROM ORDERS WHERE CUSTOMER_ID = O. CUSTOMER_ID);
```

2417 • **SELECT O.ORDER_ID,O.CUSTOMER_ID,O.TOTAL_AMOUNT AS 'HIGHEST ORDER VALUE' FROM ORDERS O**
2418 **WHERE TOTAL_AMOUNT = (SELECT MAX(TOTAL_AMOUNT) FROM ORDERS WHERE CUSTOMER_ID = O.CUSTOMER_ID);**

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	ORDER_ID	CUSTOMER_ID	HIGHEST ORDER VALUE
▶	18	13	32001.24
	33	8	36147.09
	62	10	35723.17
	66	27	26176.05
	68	24	36159.92
	80	12	38656.26
	107	14	48042.06
	121	16	37415.67

ORDERS 52 x

7. Highest Order Per Customer (Including Names).

```
SELECT C. NAME, O. ORDER_ID, O. TOTAL_AMOUNT AS 'HIGHEST ORDER' FROM CUSTOMERS C
```

```
JOIN ORDERS O ON C. CUSTOMER_ID=O.CUSTOMER_ID WHERE O. TOTAL_AMOUNT =  
(SELECT MAX(TOTAL_AMOUNT) FROM ORDERS WHERE CUSTOMER_ID = O.  
CUSTOMER_ID);
```

2422 • `SELECT C.NAME,O.ORDER_ID,O.TOTAL_AMOUNT AS 'HIGHEST ORDER' FROM CUSTOMERS C JOIN ORDERS O`
2423 `ON C.CUSTOMER_ID=O.CUSTOMER_ID WHERE O.TOTAL_AMOUNT = (SELECT MAX(TOTAL_AMOUNT) FROM ORDERS`
2424 `WHERE CUSTOMER_ID = O.CUSTOMER_ID);`

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	NAME	ORDER_ID	HIGHEST ORDER
▶	Thomas Owens	278	32015.16
	Charles Grant	315	42056.04
	Kaitlin Richards	281	41679.11
	Christina Williams	386	25747.34
	David Allen	266	39921.78
	Mark Duke	330	39003.19
	Briana Wright	128	28589.04
	John Bryan	33	36147.09

Result 53 x

LEVEL 6: SET OPERATIONS

1. List all customers who have either placed an order or written a product review.

```
SELECT CUSTOMER_ID, NAME, EMAIL FROM CUSTOMERS WHERE CUSTOMER_ID  
IN (SELECT CUSTOMER_ID FROM ORDERS UNION SELECT CUSTOMER_ID FROM  
PRODUCT_REVIEWS);
```

2432 • **SELECT** CUSTOMER_ID, **NAME**, EMAIL **FROM** CUSTOMERS **WHERE** CUSTOMER_ID **IN**
2433 **(SELECT** CUSTOMER_ID **FROM** ORDERS **UNION SELECT** CUSTOMER_ID **FROM** PRODUCT_REVIEWS);

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	CUSTOMER_ID	NAME	EMAIL
▶	1	Thomas Owens	user1@example.com
	2	Charles Grant	user2@example.com
	3	Kaitlin Richards	user3@example.com
	4	Christina Williams	user4@example.com
	5	David Allen	user5@example.com
	6	Mark Duke	user6@example.com
	7	Briana Wright	user7@example.com
	8	John Bryan	user8@example.com
	9	Jason Thompson	user9@example.com

CUSTOMERS 54 x

2. List all customers who have placed an order as well as reviewed a product.

```
SELECT CUSTOMER_ID, NAME, EMAIL FROM CUSTOMERS WHERE CUSTOMER_ID  
IN (SELECT CUSTOMER_ID FROM ORDERS) AND CUSTOMER_ID IN (SELECT  
CUSTOMER_ID FROM PRODUCT_REVIEWS);
```

2437 • **SELECT** CUSTOMER_ID, **NAME**, EMAIL **FROM** CUSTOMERS **WHERE** CUSTOMER_ID **IN** (SELECT CUSTOMER_ID **FROM** ORDERS)
2438 **AND** CUSTOMER_ID **IN** (SELECT CUSTOMER_ID **FROM** PRODUCT_REVIEWS);

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	CUSTOMER_ID	NAME	EMAIL
▶	1	Thomas Owens	user1@example.com
	2	Charles Grant	user2@example.com
	4	Christina Williams	user4@example.com
	6	Mark Duke	user6@example.com
	7	Briana Wright	user7@example.com
	9	Jason Thompson	user9@example.com
	10	Shawn Hill	user10@example.com
	11	Walter Jenkins	user11@example.com
	13	Leslie Wilson	user13@example.com

CUSTOMERS 55 x